Υπολογιστική Νοημοσύνη

Κωνσταντίνος Δεδικούσης 2962
Νικόλαος Σκρέτας 4493
Εμμανουηλίδης Εμμανουήλ 4669

Άσκηση 1)

- Εντολές μεταγλώτησης και δημιουργίας ετκελέσιμου

  PT3:
  gcc assignment1.c -o assignment1 -lm
  execute Command : ./assignment1

- Δημιουργία παραδειγμάτων

  Για την δημιουργία παραδειγμάτων γράψαμε ένα script το οποίο δημιουργεί 2 αρχεία (.csv) ένα με 4000 παραδείγματα για το σύνολο εκπαίδευσης και ένα ακόμα για το σύνολο ελέγχου.

  Δημιουργεί τυχάιους αριθμους μεταξύ του -1 και του 1 (χ1,χ2) και σύμφωνα με τις οδηγίες τους κατατάσσει στις κατηγορίες (C1,C2,C3,C4) και αποθηκεύει το κάθε παράδειγμα (χ1,χ2,class) στο εκάστοτε αρχείο.

- Υλοποίηση της άσκησης

```
8   #define D 2
9   #define K 4
10  #define H1 30
11  #define H2 15
12  #define H3 10
13  #define ACTIVATION_LOGISTIC 2
14  #define ACTIVATION_RELU 1
15  #define ACTIVATION_TANH 0
16  #define ACT_H1 ACTIVATION_TANH
17  #define ACT_H2 ACTIVATION_TANH
18  #define ACT_H3 ACTIVATION_TANH
19  #define MIN_EPOCHS 800
20  #define MAX_ERROR 0.0001
21  #define LEARNING_RATE 0.001
22  #define N 4000
23  #define B 200
```

1.
2. Για το φόρτωμα των συνόλων χρησιμοποιήσαμε τα 2 προαναφερθέντα αρχεία και τα φορτώσαμε σε ένα FILE αντικείμενο

   Παράλληλα δημιουργήσαμε ένα struct examples το οποίο κρατάει την πληροφορία για κάθε ζεύγος παραδείγματος σε έναν πίνακα και έναν ακόμα πίνακα την πληροφορία για την επιθυμητή έξοδο με 0 για κάθε θέση που δεν αντιστοιχεί στην κατηγορία του και 1 στην θέση που αντιστοιχεί.

   Με την φόρτωση του αρχείου κάθε γραμμή δημιουργεί ένα “examples” και το αποθηκεύει σε έναν πίνακα από examples από όπου λαμβάνουμε τα στοιχεία που χρησιμοποιούνται στην ροή του προγράμματος

3. Ο καθορισμός του κατωφλιού και του ρυθμού μάθησης φαίνεται από την εικόνα του ερωτήματος 1.

   Ορισμός απαιτούμενων πινάκων και δομών:

```
25    // Weights and Biases for each level
26    // Inp -> H1
27    double weights_inp_to_h1[D][H1];
28    double der_errors_w_inp_to_h1[D][H1];
29
30    double bias_h1[H1];
31    double der_errors_b_h1[H1];
32
33    double h1_total_inputs[H1];
34    double h1_outputs[H1];
35
36    // H1 -> H2
37    double weights_h1_to_h2[H1][H2];
38    double der_errors_w_h1_to_h2[H1][H2];
39
40    double bias_h2[H2];
41    double der_erros_b_h2[H2];
42
43    double h2_total_inputs[H2];
44    double h2_outputs[H2];
45
46    // H2 -> H3
47    double weights_h2_to_h3[H2][H3];
48    double der_errors_w_h2_to_h3[H2][H3];
49
50    double bias_h3[H3];
51    double der_erros_b_h3[H3];
52
53    double h3_total_inputs[H3];
54    double h3_outputs[H3];
55
56    // H3 -> Out
57    double weights_h3_to_out[H3][K];
58    double der_errors_w_h3_to_out[H3][K];
59
60    double bias_out[K];
61    double der_errors_b_out[K];
62
63    double outs[K];
64    double *outputs;
```

Στην ουσία δημιουργουμε για μελλοντική χρήση σε επίπεδο

1 Πίνακα για τα βάρη από τι H-1 στο H

1 Πίνακα για τις παραγώγους των βαρών
1 Πίνακα για τις πολώσεις του Η
1 Πίνακα για τις εισόδους του Η
1 Πίνακα για τις εξόδους του Η

Και για την τυχαία αρχικοποίηση των βαρών και των πολώσεων στο [-1,1]

```
420         //***** initialize weights and biases *****
421
422         // weights D to H1 and biases of H1
423         for (int i = 0; i < H1; i++)
424         {
425             bias_h1[i] = make_random_in_range();
426             for (int j = 0; j < D; j++)
427             {
428                 weights_inp_to_h1[j][i] = make_random_in_range();
429             }
430         }
431
432         // weights H1 to H2 and biases of H2
433         for (int i = 0; i < H2; i++)
434         {
435             bias_h2[i] = make_random_in_range();
436             for (int j = 0; j < H1; j++)
437             {
438                 weights_h1_to_h2[j][i] = make_random_in_range();
439             }
440         }
441
442         // weights H2 to H3 and biases of H3
443         for (int i = 0; i < H3; i++)
444         {
445             bias_h3[i] = make_random_in_range();
446             for (int j = 0; j < H2; j++)
447             {
448                 weights_h2_to_h3[j][i] = make_random_in_range();
449             }
450         }
451
452         // weights H3 to out and biases of out
453         for (int i = 0; i < K; i++)
454         {
455             bias_out[i] = make_random_in_range();
456             for (int j = 0; j < H3; j++)
457             {
458                 weights_h3_to_out[j][i] = make_random_in_range();
459             }
460         }
```

4.Υλοποίηση της forward_pass

```
248  double *forward_pass(double *x, int d, double *y, int k)
249  {
250
251      // From inputs to H1
252      for (int i = 0; i < H1; i++)
253      {
254          double temp = 0;
255          for (int j = 0; j < d; j++)
256          {
257              temp += x[j] * weights_inp_to_h1[j][i];
258          }
259          temp += bias_h1[i];
260          h1_total_inputs[i] = temp;
261          h1_outputs[i] = func_activation(temp, ACT_H1);
262      }
263
264      // From H1 to H2
265      for (int i = 0; i < H2; i++)
266      {
267          double temp = 0;
268          for (int j = 0; j < H1; j++)
269          {
270              temp += h1_outputs[j] * weights_h1_to_h2[j][i];
271          }
272          temp += bias_h2[i];
273          h2_total_inputs[i] = temp;
274          h2_outputs[i] = func_activation(temp, ACT_H2);
275      }
276
277      // From H2 to H3
278      for (int i = 0; i < H3; i++)
279      {
280          double temp = 0;
281          for (int j = 0; j < H2; j++)
282          {
283              temp += h2_outputs[j] * weights_h2_to_h3[j][i];
284          }
285          temp += bias_h3[i];
286          h3_total_inputs[i] = temp;
287          h3_outputs[i] = func_activation(temp, ACT_H3);
288      }
289
290      // From H3 to Out
291      for (int i = 0; i < k; i++)
292      {
293          double temp = 0;
294          for (int j = 0; j < H3; j++)
```

```
295             {
296                 temp += h3_outputs[j] * weights_h3_to_out[j][i];
297             }
298             temp += bias_out[i];
299             outs[i] = temp;
300         }
301         y = output_function(outs, k);
302 
303         return y;
304     }
```

Η υλοποιήση της συνάρτησης πραγματοποιείται ως εξής,

Αρχικά η κάθε είσοδος δίνει πληροφορία σε κάθε νευρώνα του επόμενου επιπέδου όπου πολλαπλασιάζοντας με το βάρος της διαδρομής και προσθέτοντας την πόλωση έχουμε την είσοδο του νευρώνα οπου αποθηκευεται κατάλληλα (για χρήση αργότερα) και με χρήση της συνάρτησης ενεργοποίησής (relu η tanhu) παίρνουμε την έξοδο του νευρώνα.

Επαναλαμβάνουμε για κάθε κρυμμένο επίπεδο ενώ για την έξοδο ακολουθούμε τα ίδια βήματα έως την χρήση της συνάρτησης ενεργοποίησης όπου στην προκειμένη φάση χρησημοποιούμε την συνάρτηση softmax (για να φτιάξουμε ποσοστά από τις εισόδους των νευρώνων εξόδου) όπου μας δίνει το διάνυσμα εξόδου (K1,K2,K3,K4) με τιμές μεταξύ του (0,1) και Σ(K) = 1 ώστε να λαμβάνουμε ώς κατηγορία την έξοδο με τιμή πλησιέστερη στο 1.

5.Υλοποίηση της backprop

```
306  void backprop(double *x, int d, double *t, int k)
307  {
308      // ---------- Output layer deltas ----------
309      double delta_out[K];
310      for (int i = 0; i < K; i++)
311      {
312          delta_out[i] = outputs[i] - t[i]; // softmax + cross-entropy gradient
313          der_errors_b_out[i] = delta_out[i];
314          for (int j = 0; j < H3; j++)
315          {
316              der_errors_w_h3_to_out[j][i] = h3_outputs[j] * delta_out[i];
317          }
318      }
319
320      // ---------- H3 layer deltas ----------
321      double delta_H3[H3];
322      for (int i = 0; i < H3; i++)
323      {
324          double temp = 0.0;
325          for (int j = 0; j < K; j++)
326          {
327              temp += weights_h3_to_out[i][j] * delta_out[j]; // i < H3 OK
328          }
329          delta_H3[i] = derivative_activation(h3_total_inputs[i], ACT_H3) * temp;
330          der_erros_b_h3[i] = delta_H3[i];
331
332          for (int j = 0; j < H2; j++)
333          {
334              der_errors_w_h2_to_h3[j][i] = h2_outputs[j] * delta_H3[i];
335          }
336      }
337
338      // ---------- H2 layer deltas ----------
339      double delta_H2[H2];
340      for (int i = 0; i < H2; i++)
341      {
342          double temp = 0.0;
343          for (int j = 0; j < H3; j++)
344          {
345              temp += weights_h2_to_h3[i][j] * delta_H3[j];
346          }
347          delta_H2[i] = derivative_activation(h2_total_inputs[i], ACT_H2) * temp;
348          der_erros_b_h2[i] = delta_H2[i];
349
350          for (int j = 0; j < H1; j++)
351          {
352              der_errors_w_h1_to_h2[j][i] = h1_outputs[j] * delta_H2[i];
```

```
353          }
354      }
355
356      // ---------- H1 layer deltas ----------
357      double delta_H1[H1];
358      for (int i = 0; i < H1; i++)
359      {
360          double temp = 0.0;
361          for (int j = 0; j < H2; j++)
362          {
363              temp += weights_h1_to_h2[i][j] * delta_H2[j];
364          }
365          delta_H1[i] = derivative_activation(h1_total_inputs[i], ACT_H1) * temp;
366          der_errors_b_h1[i] = delta_H1[i];
367
368          for (int j = 0; j < D; j++)
369          {
370              der_errors_w_inp_to_h1[j][i] = x[j] * delta_H1[i];
371          }
372      }
373  }
```

Αντίστροφα από την forwardpass για την backprop ξεκινάμε από τους νευρώνες εξόδου και υπολογίζουμε τα σφάλματα (δ) και τις μερικές παραγώγους βάρους σύνδεσης (και πολώσεων) σύμφωνα με τους τύπους για το αντίστροφο πέρασμα (Κ4 διαφάνεις μαθήματος) όπου και τα αποθηκεύουμε σε έναν πίνακα (για χρήση στην gradient descent). Επαναλαμβάνουμε μέχρι να φτάσουμε στο επίπεδο εισόδου.

6.Υλοποίηση gradient descent δημιουργούμε για while(true) loop όπου και μέσα εκτελείται η βασική ροή του προγράμματος Και «σπάει» (break;) αν η τρέχουσα εποχή είναι μεγαλύτερη από 800 και το απόλυτο της διαφοράς του συνολικού σφάλματος εκπαίδευσης 2 διαδοχικών εποχών είναι μικρότερο του κατωφλιού τερματισμού.
Για N/B batches δημιουργούμε προσωρινούς πίνακες για αποθήκευση τιμών παραγώγων των βαρών (ώστε να μπορούμε να έχουμε την πληροφορία της τρέχουσας εποχής) και για κάθε batch (δηλ για Β φορές και με αυτόν τον τρόπου επιτυγχάνουμε να μπορούμε να χρησημιοποιύμε batches με διαφορετικό μέγεθος κάθε φορά που τρέχει το πρόγραμμα), υλοποιούμαι forward_pass και backprop όπου ενημερώνονται οι πίνακες των σφαλμάτων/παραγώγων. Έπειτα σύμφωνα με τους τύπους ενημερώνουμε τα βάρη και τις πολώσεις βρίσκουμε το συνολικό σφάλμα της εποχής και το τυπώνουμε.

7. Αφού τερματίσει η εκπαίδευση ελέγχουμε την ικανότητα γενίκευσης του προγράμματος υλοποιώντας με τα πλέον

ανανεωμένα βάρη την forward_pass ,για κάθε παράδειγμα του συνόλου ελέγχου, η οποία μας δίνει έναν πίνακα από τον οποίο λαμβάνουμε το στοιχείο με την μέγιστη τιμή όπου και αυτό το στοιχείο είναι και η κατηγορία στην οποία ταξινομείται. Συγκρίνουμε το αποτέλεσμα για κάθε παράδειγμα με την επιθυμητή έξοδο του και ελέγχοντας αν συμβαδίζουν υπολογίζουμε το ποσοστό σωστών αποφάσεων στο σύνολο ελέγχου το όποιο και τυπώνουμε.

Εξέταση και καταγραφή αποτελεσμάτων:

H1	H2	H3	activation 3	B = N/10 (400)	B = N/20 (200)	B= N/100 (40)	B = N/200 (20)
20	10	10	tan	88.08 %	90.17 %	90.17 %	92.75 %
20	10	10	logistic	84.85 %	91.88 %	87.55 %	90.20 %
20	10	10	relu	84.25 %	85.50 %	82.97 %	90.28 %
30	15	10	tan	90.65 %	93.17 %	92.55 %	92.05 %
30	15	10	logistic	89.80 %	91.70 %	91.90 %	93.00 %
30	15	10	relu	78.08 %	84.28 %	88.67 %	90.53 %
15	5	5	tan	80.50 %	67.85 %	71.28 %	74.03 %
15	5	5	logistic	61.00 %	72.33 %	64.33 %	76.67 %
15	5	5	relu	58.65 %	67.80 %	85.75 %	77.28 %

Από τον πίνακα αποτελεσμάτων φαίνεται ότι η απόδοση του MLP επηρεάζεται σημαντικά τόσο από την αρχιτεκτονική του δικτύου όσο και από τη συνάρτηση ενεργοποίησης και το μέγεθος του batch. Τα μικρά δίκτυα (π.χ. 15–5–5) παρουσιάζουν χαμηλά ποσοστά επιτυχίας, κάτι που δείχνει ότι δεν έχουν αρκετή πολυπλοκότητα για να μάθουν σωστά το πρόβλημα.

Όσο αυξάνεται ο αριθμός των νευρώνων στα κρυμμένα επίπεδα, η ακρίβεια βελτιώνεται αισθητά, με το δίκτυο 30–15–10 να δίνει τα καλύτερα συνολικά αποτελέσματα. Όσον αφορά τις συναρτήσεις ενεργοποίησης, η tanh φαίνεται να αποδίδει καλύτερα και πιο σταθερά σε όλες τις περιπτώσεις, ενώ η logistic έχει ελαφρώς χαμηλότερη απόδοση. Η relu εμφανίζει πιο ασταθή συμπεριφορά, ειδικά στα μικρότερα δίκτυα.

Τέλος, παρατηρείται ότι η αύξηση του μεγέθους του batch οδηγεί γενικά σε καλύτερα αποτελέσματα, καθώς η εκπαίδευση γίνεται πιο σταθερή. Συνολικά, ο καλύτερος συνδυασμός για το συγκεκριμένο πρόβλημα είναι ένα μεγαλύτερο δίκτυο με συνάρτηση ενεργοποίησης tanh και σχετικά μεγάλο batch.

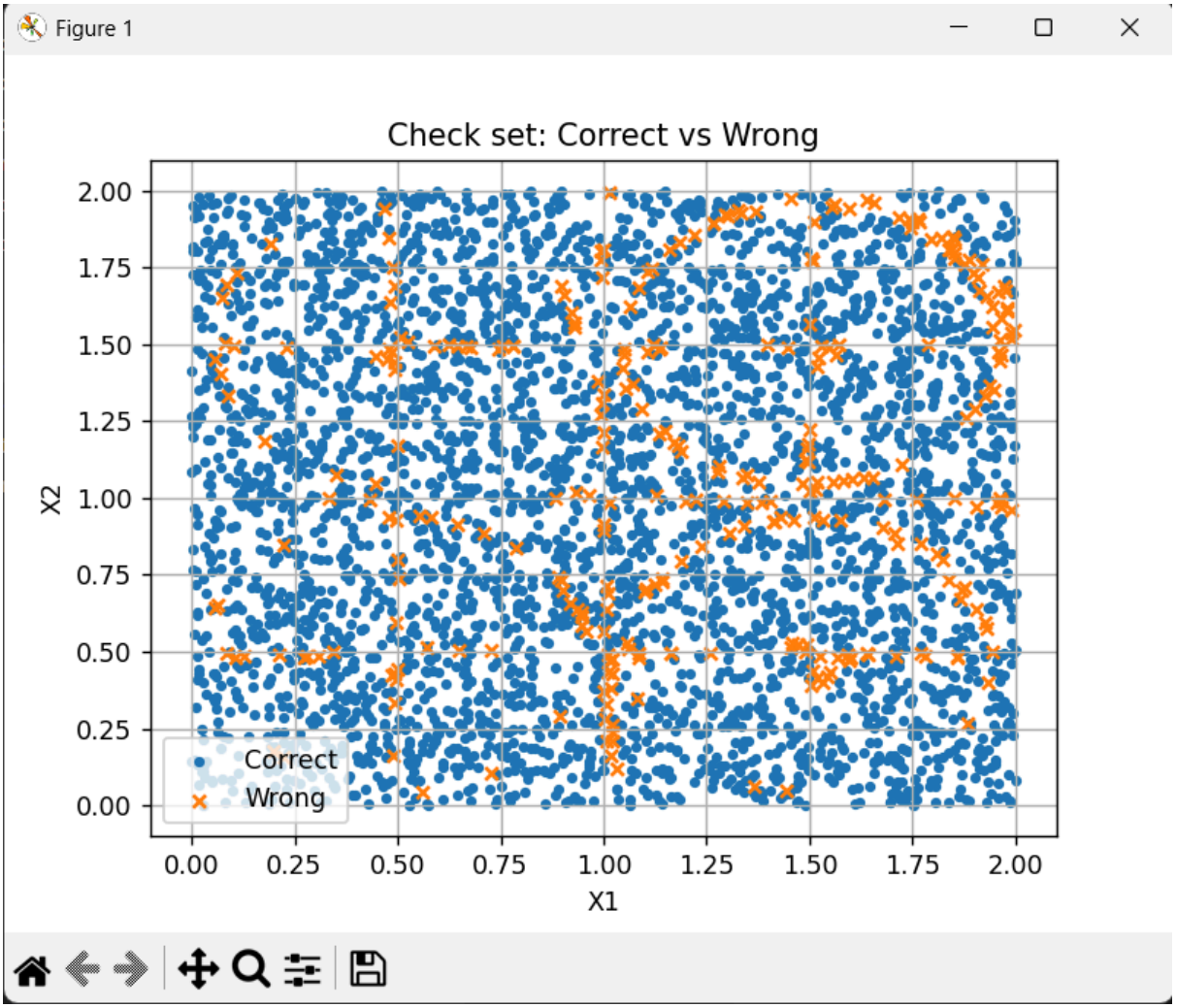
Εκτύπωση Παραδειγμάτων:
Ενδεικτικό screenshot

```
3972. X1 =  1.429276, X2 =  1.896469  cl = 1  (+)
3973. X1 =  1.697228, X2 =  1.714853  cl = 0  (+)
3974. X1 =  1.164254, X2 =  0.090015  cl = 3  (+)
3975. X1 =  0.992211, X2 =  1.103987  cl = 3  (+)
3976. X1 =  1.011063, X2 =  0.690027  cl = 3  (-)
3977. X1 =  0.316394, X2 =  1.580858  cl = 1  (+)
3978. X1 =  1.451931, X2 =  1.281907  cl = 0  (+)
3979. X1 =  0.273583, X2 =  0.391129  cl = 0  (+)
3980. X1 =  1.909232, X2 =  0.403873  cl = 1  (+)
3981. X1 =  0.885314, X2 =  0.868655  cl = 2  (+)
3982. X1 =  1.544711, X2 =  0.091994  cl = 1  (+)
3983. X1 =  0.620756, X2 =  1.190403  cl = 1  (+)
3984. X1 =  0.558978, X2 =  0.178538  cl = 1  (+)
3985. X1 =  1.099343, X2 =  0.696623  cl = 1  (-)
3986. X1 =  0.831218, X2 =  1.964091  cl = 3  (+)
3987. X1 =  1.732220, X2 =  1.577282  cl = 0  (+)
3988. X1 =  1.332463, X2 =  0.538628  cl = 1  (+)
3989. X1 =  0.569454, X2 =  0.512129  cl = 0  (-)
3990. X1 =  1.285222, X2 =  1.745251  cl = 1  (+)
3991. X1 =  1.460108, X2 =  0.930546  cl = 1  (-)
3992. X1 =  1.465187, X2 =  1.755102  cl = 1  (+)
3993. X1 =  1.719975, X2 =  0.729010  cl = 0  (+)
3994. X1 =  0.207618, X2 =  0.895050  cl = 2  (+)
3995. X1 =  0.397640, X2 =  0.343877  cl = 0  (+)
3996. X1 =  1.410029, X2 =  0.094033  cl = 0  (+)
3997. X1 =  1.546388, X2 =  1.705303  cl = 0  (+)
3998. X1 =  1.231773, X2 =  0.714371  cl = 1  (+)
3999. X1 =  0.952713, X2 =  0.269303  cl = 2  (+)
4000. X1 =  0.459055, X2 =  0.524207  cl = 1  (+)
```

Επίσης, για την παραγωγή του γραφήματος χρησιμοποιήθηκαν τα αρχεία `check_correct` και `check_wrong`, τα οποία δημιουργούνται κατά τον έλεγχο του δικτύου στο σύνολο ελέγχου. Για κάθε δείγμα του check set, το πρόγραμμα ελέγχει αν η έξοδος του MLP είναι σωστή ή λανθασμένη και αποθηκεύει τις αντίστοιχες τιμές των χαρακτηριστικών εισόδου (X1, X2) στο αντίστοιχο αρχείο.

Στη συνέχεια, μέσω ενός ξεχωριστού script, τα δεδομένα των δύο αρχείων διαβάζονται και απεικονίζονται σε κοινό γράφημα. Τα σημεία του αρχείου `check_correct` εμφανίζονται ως σωστές ταξινομήσεις, ενώ τα σημεία του `check_wrong` ως

λανθασμένες. Με αυτόν τον τρόπο προκύπτει το τελικό plot που χρησιμοποιείται για την οπτικοποίηση της απόδοσης του ταξινομητή.



Από την οπτικοποίηση των αποτελεσμάτων στο σύνολο ελέγχου προκύπτει ότι το πολυεπίπεδο perceptron παρουσιάζει πολύ καλή γενίκευση, καθώς η συντριπτική πλειονότητα των δειγμάτων ταξινομείται σωστά. Τα σφάλματα δεν εμφανίζονται τυχαία, αλλά εντοπίζονται κυρίως κοντά στα όρια απόφασης μεταξύ των κλάσεων, όπου η ταξινόμηση είναι εγγενώς πιο δύσκολη.

Η χωρική κατανομή των λαθών υποδηλώνει ότι το δίκτυο έχει μάθει τη βασική δομή του προβλήματος και προσεγγίζει

ικανοποιητικά τη μη γραμμική επιφάνεια απόφασης. Οι αποκλίσεις που παρατηρούνται μπορούν να αποδοθούν στην πολυπλοκότητα του προβλήματος, στον πεπερασμένο αριθμό νευρώνων ή/και στις παραμέτρους εκπαίδευσης.

Συνολικά, τα αποτελέσματα κρίνονται ικανοποιητικά, ενώ πιθανές βελτιώσεις θα μπορούσαν να επιτευχθούν με ρύθμιση της αρχιτεκτονικής του δικτύου, του ρυθμού μάθησης ή του αριθμού εποχών εκπαίδευσης.